¡Linux File and Directory Commands, File Permissions, Ownership, and Process Monitoring

1. Aim

To study and perform basic Linux command-line operations, including directory navigation, file creation, modification, copying, moving and deletion. To understand and implement file access permissions using `chmod`, manage file ownership using `chown`, and monitor running processes using Linux process-monitoring utilities.

2. Theory

2.1 Introduction to Linux

Linux is an open-source, Unix-like operating system that manages computer hardware and software resources. It provides a command-line interface (CLI) through which users can interact with the operating system by executing commands.

The Linux CLI allows users to perform operations such as:

- Creating and managing files and directories.
- Changing file permissions and ownership.
- Monitoring running processes.
- Managing system resources.
- Executing administrative commands.

The Linux shell acts as an interface between the user and the operating system.

2.2 Linux File System

Linux uses a hierarchical file system structure that begins with the root directory `/`.

Important directories include:

Directory	Purpose

`/`	Root directory
`/home`	User home directories
`/root`	Root user's home directory
`/bin`	Essential user commands
`/etc`	System configuration files
`/var`	Variable data and logs
`/tmp`	Temporary files
`/usr`	User applications and utilities

2.3 Basic Linux Navigation Commands

Linux provides several commands to navigate and explore directories.

Command	Description
`pwd`	Displays the current working directory
`ls`	Lists files and directories

`ls -la`	Displays all files with detailed information
`cd`	Changes the current directory
`cd ..`	Moves to the parent directory
`cd ~`	Moves to the user's home directory
`clear`	Clears the terminal screen

2.4 File and Directory Management

Linux provides commands to create, view, copy, rename, move and delete files and directories.

Command	Description
`mkdir`	Creates a directory
`rmdir`	Removes an empty directory
`touch`	Creates an empty file
`cat`	Displays file contents
`nano`	Opens a text editor

`cp`	Copies files
`mv`	Moves or renames files
`rm`	Deletes files
`rm -r`	Removes directories recursively

Example:

```
None
mkdir Linux_Lab
cd Linux_Lab
touch file1.txt
echo "Welcome to Linux" > file1.txt
cat file1.txt
```

2.5 Linux File Permissions

Linux uses a permission system to control access to files and directories.

There are three types of permissions:

Permission	Symbol	Numeric Value	Meaning
Read	r	4	Read file contents
Write	w	2	Modify file contents

Execute	x	1	Execute a file or access a directory

Permissions are assigned to three categories:

1. User (Owner): The person who owns the file.
2. Group: Users belonging to the file's group.
3. Others: All remaining users.

For example:

```
None
-rwxr-xr--
```

This represents:

- Owner: Read, Write, Execute.
- Group: Read, Execute.
- Others: Read only.

2.6 chmod Command

The `chmod` command is used to change file and directory permissions.

Syntax:

```
None
chmod permissions filename
```

Example:

```
None
chmod 755 file1.txt
```

Permission 755 means:

- Owner = 7 (4+2+1)
- Group = 5 (4+1)
- Others = 5 (4+1)

Therefore, the owner has full permissions, while the group and others have read and execute permissions.

2.7 chown Command

The `chown` command is used to change the ownership of a file or directory.

Syntax:

```
None
sudo chown username filename
```

To change both owner and group:

```
None
sudo chown username:groupname filename
```

Only authorized users, generally root or users with appropriate privileges, can change ownership.

2.8 Process Monitoring in Linux

A process is an instance of a running program. Every process has a unique Process ID (PID).

Linux provides various commands to monitor and manage processes.

Command	Description
`ps`	Displays running processes
`ps aux`	Displays detailed process information
`top`	Displays real-time process information

`htop`	Provides an interactive process monitor, if installed
`pidof`	Displays the PID of a running program
`kill`	Sends a signal to a process
`jobs`	Displays background jobs
`sleep`	Delays execution for a specified time

The `top` command helps monitor:

- CPU utilization.
- Memory utilization.
- Running processes.
- Process IDs.
- System load.

2.9 Importance of Linux Commands

Linux commands are important because they help users manage files, control access permissions, monitor system performance and automate administrative operations efficiently.

3. Code / Commands

The following commands demonstrate all the operations covered in this practical.

Complete command sequence:

```
None
# Step 1: Display current working directory
pwd

# Step 2: Display files and directories
```

```
ls
ls -la

# Step 3: Create a practical directory
mkdir Linux_Lab

# Step 4: Enter the directory
cd Linux_Lab

# Step 5: Create files
touch file1.txt file2.txt file3.txt

# Step 6: Write content into file1.txt
echo "Welcome to Linux Operating System Lab" > file1.txt

# Step 7: Display file contents
cat file1.txt

# Step 8: Copy a file
cp file1.txt file2.txt

# Step 9: Rename a file
mv file2.txt renamed_file.txt

# Step 10: List all files
ls -l

# Step 11: Change file permissions
chmod 755 file1.txt

# Step 12: Display file permissions
ls -l file1.txt

# Step 13: Display current username
whoami
```

```
# Step 14: Display running processes
ps aux

# Step 15: Start a background process
sleep 300 &

# Step 16: Display background jobs
jobs

# Step 17: Find the sleep process
ps aux | grep '[s]leep 300'

# Step 18: Display real-time process monitoring
top

# Step 19: Terminate the sleep process
pkill -f '^sleep 300$'

# Step 20: Remove unwanted files
rm renamed_file.txt
rm file2.txt
rm file3.txt

# Step 21: Return to parent directory
cd ..

# Step 22: Display directory contents
ls -la
```

Note: Run the commands sequentially. Press Q to exit the top command. The pkill command terminates the background sleep process started in Step 15.

4. Execution Instructions

Requirements

- Operating System: Ubuntu Linux / Kali Linux.

- Terminal: Linux Terminal.
- Shell: Bash.
- User account with appropriate permissions.

Step 1: Open Linux Terminal

Press `Ctrl + Alt + T` in Ubuntu to open the terminal.

Terminal - an interface for command-line operations - Data Science Workbook
datascience.101workbook.org

Step 2: Create a working directory

Execute:

```
None
mkdir Linux_Lab
cd Linux_Lab
```

Step 3: Create and manage files

Execute:

```
None
touch file1.txt file2.txt file3.txt
echo "Welcome to Linux Lab" > file1.txt
cat file1.txt
```

Step 4: Change file permissions

Execute:

```
None
chmod 755 file1.txt
ls -l file1.txt
```

Verify that the permissions are displayed as:

```
None
-rwxr-xr-x
```

Step 5: Monitor running processes

Execute:

```
None
sleep 300 &
ps aux | grep '[s]leep 300'
top
```

Observe the running process and its resource utilization.

Step 6: Terminate the process

Execute:

```
None
pkill -f '^sleep 300$'
```

The background process will be terminated.

Step 7: Verify the operations

Execute:

```
None
ls -la
```

Check the files and permissions.

5. Result

The following operations were demonstrated using Linux command-line utilities:

1. Successfully navigated through Linux directories.
2. Created and managed files and directories.
3. Performed file copying, renaming and deletion.

4. Changed file permissions using the `chmod` command.
5. Studied file ownership management using `chown`.
6. Monitored running processes using `ps` and `top`.
7. Managed background processes using process-control commands.

6. Conclusion

In this practical, basic Linux command-line operations were studied and demonstrated successfully. File and directory management commands were used to create, modify, copy and delete files. File access permissions were understood and modified using the `chmod` command, while ownership management was studied using `chown`. Process monitoring utilities such as `ps`, `top` and `kill` were also explored. Thus, the practical provided a fundamental understanding of Linux file management, access control and process monitoring.